

Safer Bedienungsdokumentation

Version: 1.0

Datum: September 2026

Autor: Peter Boettcher

Lizenz: GPLv2

Inhaltsverzeichnis

1. Überblick
2. Installation
3. Die Konfigurationsdatei
4. Das bsafes.sh-Skript
5. Workflows
6. /proc-Auswertung
7. Container und Host
8. Glossar
9. Sicherheitshinweise
10. Abschließende Zusammenfassung

1. Überblick

Safer ist ein **Kernel-basiertes Zugriffskontrollsystem**, das folgende Hauptfunktionen bietet:

Funktion	Beschreibung
Programmprüfung	Prüfung von Programmen anhand von Pfad, SHA256-Hash und UID/GID
Skriptprüfung	Prüfung von Skripten MIT und OHNE Shebang anhand von Hash
Interpreter-Prüfung	Interpreter (ai:) dürfen NUR in Verbindung mit erlaubten Skripten laufen
Bibliotheksprüfung	Prüfung von Shared Objects, anhand von Hash
Kernel-Modul-Prüfung	Deaktivierung von <code>init_module</code> , Prüfung erlaubter Module
Shell-Inline-Schutz	Blockierung von Shell-Inline-Code (SHELL-INLINE:)
Konfigurationsdatei-Schutz	Inhaltliche Prüfung (KONFIG_SFILE:) und Hash-Prüfung (KONFIG:)
Container-Schutz	Kontexterkennung Host/Container, unterschiedliche Reaktionen
User/Group-Verwaltung	UID/GID-basierte Berechtigungssteuerung für Programme
Learning Mode	Automatische Generierung der Allow Liste (SLON/SLOFF)
Boot-Schutz	Initramfs und Kernel-Integration (Möglich)
Statische Integration	Code fest im Kernel einkompiliert, Allow Liste möglich

2. Installation

2.1 Kernel-Kompilierung

2.2 initramfs-Integration (Möglich, für Boot-Schutz)

2.2 Integration Listen in Kernel (Möglich, für Boot-Schutz)

3. Die Konfigurationsdatei

3.1 Dateiformate

Programme (a:) – User-basiert

a:<UID>;<Größe>;<SHA256-Hash>;<Pfad>

Skripte (a:) – User-basiert (MIT und OHNE Shebang)

a:<UID>;<Größe>;<SHA256-Hash>;<Pfad>

Programme verbieten (d:) – User-basiert

d:<UID>;<Größe>;<SHA256-Hash>;<Pfad>

Programme (a:*) – Alle User

a:*<Größe>;<SHA256-Hash>;<Pfad>

Programme verbieten (d:*) – Alle User

d:*<Größe>;<SHA256-Hash>;<Pfad>

Ordner verbieten (d:*) – Alle User

d: <Order werden durch (/) am Zeilenende erkannt>

Interpreter (ai:) – NUR in Verbindung mit Skripten

ai:<UID>;<Größe>;<SHA256-Hash>;<Pfad>

Shells (as:) – Fuer Inline-Code

as:<UID>;<Größe>;<SHA256-Hash>;<Pfad>

Globale Programme (ga:) – Gruppen-basiert

ga:<GID>;<Größe>;<SHA256-Hash>;<Pfad>

Bibliotheken (so:) – Keine User/Group-Bindung

so:<Größe>;<SHA256-Hash>;<Pfad>

Kernel-Module (ko:) – Keine User/Group-Bindung

ko:<Größe>;<SHA256-Hash>;<Pfad>

Konfigurationsdateien (KONFIG:) – Hash-Prüfung (NUR Host)

KONFIG:<Größe>;<SHA256-Hash>;<Pfad>

Shell-Inline-Muster (SHELL-INLINE:) – Keine User/Group-Bindung

SHELL-INLINE:<Muster>

3.2 Beispiel-Konfiguration

```
# =====
# INTERPRETER (ai:) – NUR in Verbindung mit Skripten
# =====
ai:1000;6828688;HASH;/usr/bin/python3
ai:0;3931280;HASH;/usr/bin/perl
ai:0;14384;HASH;/usr/bin/java
```

Verhindert das ld-linux-x86-64.so.2 etc. als Loader vom USERLAND genutzt wird

```
ai:*;225600;HASH;/lib64/ld-linux-x86-64.so.2
```

Im Container eine andere ld-linux-.. Datei, möglich

```
ai:0;214600;HASH;/lib64/ld-linux-x86-64.so.2
```

```
ai:1000;225600;HASH;/lib64/ld-linux-x86-64.so.2
```

```
ai:6;225600;HASH;/lib64/ld-linux-x86-64.so.2
```

```
ai:7;225600;HASH;/lib64/ld-linux-x86-64.so.2
```

```
# =====
# SHELLS (as:) – Prüfung Inline-Code
# =====
as:1000;129736;HASH;/bin/sh
as:1000;1298416;HASH;/bin/bash
```

```
# =====
# SKRIPTE (a:) – MIT und OHNE Shebang (alle müssen erlaubt sein)
# =====
a:0;147372;HASH;/usr/scripts/start.osi.py
a:1000;468;HASH;/usr/scripts/user/p3.i.div.py
```

```
# =====
# SKRIPTE OHNE SHEBANG – MÜSSEN TROTZDEM ERLAUBT SEIN!
# =====
a:1000;42;HASH;/usr/scripts/hello.py
```

```
# =====
# NORMALE PROGRAMME (a:)
# =====
a:0;1298416;HASH;/bin/bash
a:1000;158632;HASH;/usr/bin/ls
```

```
# =====
# Ordner (a:) Order werden von bsafer.sh anhand des (/) am Zeilenende erkannt
# =====
a:*/proc/
a:0;proc/
a:1000;proc/
```

```
# =====  
# BIBLIOTHEKEN (so:)  
# =====  
so;100192;HASH;/usr/lib/libserd-0.so
```

```
# =====  
# KERNEL-MODULE (ko:)  
# =====  
ko;44792;HASH;/usr/lib/modules/Version/efivarfs.ko
```

```
# =====  
# KONFIGURATIONSDATEIEN (NUR Host). Zuerst dem Kernel Übergeben, dann erhält  
# man vom Kernel die Zeilen für KONFIG. Siehe: /proc/safer.learning  
# =====  
KONFIG_SFILE:/etc/passwd  
KONFIG_SFILE:/etc/shadow  
KONFIG_SFILE:/etc/group  
KONFIG_SFILE:/etc/gshadow  
KONFIG_SFILE:/etc/sudoers
```

```
# =====  
# KONFIGURATIONSDATEIEN (Hash – NUR Host)  
# =====  
KONFIG:3756;HASH;/etc/passwd  
KONFIG:3955;HASH;/etc/shadow
```

```
# =====  
# SHELL-INLINE (Admin-definierbar)  
# =====  
SHELL-INLINE:read<  
SHELL-INLINE:read|  
SHELL-INLINE:readarray  
SHELL-INLINE:readarray<  
SHELL-INLINE:mapfile
```

4. Das bsafer.sh-Skript

4.1 Hilfe anzeigen

bsafer.sh

4.2 Systemsteuerung

Befehl	Funktion
bsafer.sh SON	Safer aktivieren
bsafer.sh SOFF	Safer deaktivieren
bsafer.sh SONC	Vollständige Prüfung aktivieren
bsafer.sh SOFFC	Vollständige Prüfung deaktivieren
bsafer.sh SONCP	Vollständige Prüfung für Programme aktivieren
bsafer.sh SOFFCP	Vollständige Prüfung für Programme deaktivieren
bsafer.sh SONCL	Vollständige Prüfung für Bibliotheken aktivieren
bsafer.sh SOFFCL	Vollständige Prüfung für Bibliotheken deaktivieren
bsafer.sh SLOCK	Safer sperren (keine Änderungen mehr möglich)

4.3 Learning Mode

Befehl	Funktion
bsafer.sh SLON	Learning Mode aktivieren
bsafer.sh SLOFF	Learning Mode deaktivieren

4.4 Printk-Steuerung

Befehl	Funktion
bsafer.sh PAON	Printk ALLOW ON
bsafer.sh PAOFF	Printk ALLOW OFF
bsafer.sh PAONP	Printk ALLOW ON (Programme)
bsafer.sh PAOFFP	Printk ALLOW OFF (Programme)
bsafer.sh PDON	Printk DENY ON
bsafer.sh PDOFF	Printk DENY OFF
bsafer.sh PKON	Printk KONFIG ON
bsafer.sh PKOFF	Printk KONFIG OFF

4.5 Verbose-Mode

Befehl	Funktion
<code>bsafer.sh SVON</code>	Verbose ON (Argumente)
<code>bsafer.sh SVOFF</code>	Verbose OFF
<code>bsafer.sh SVUKON</code>	Unknown File Log ON
<code>bsafer.sh SVUKOFF</code>	Unknown File Log OFF
<code>bsafer.sh SODON</code>	Show Deny ON, Programm wird ausgeführt, obwohl Schutz aktiv. Man erhält dann übersichtlich, alle Programme, die noch nicht von Learning erfasst wurden.
<code>bsafer.sh SODOFF</code>	Show Deny OFF

4.6 Listen laden (Duplikate werden automatisch entfernt)

Befehl	Funktion
<code>bsafer.sh PLIST <datei></code>	Programm- und Skript-Allow Liste laden
<code>bsafer.sh FLIST <datei></code>	Ordner-Allow Liste laden
<code>bsafer.sh LLIST <datei></code>	Bibliotheks-Allow List laden
<code>bsafer.sh KLIST <datei></code>	Kernel-Modul-Allow List laden
<code>bsafer.sh SKONFIG-LIST <datei></code>	Konfigurationsliste laden (Host)
<code>bsafer.sh KONFIG-LIST <datei></code>	Konfigurations-Hash-Liste laden (Host)
<code>bsafer.sh KONFIG-PATTERN-LIST <datei></code>	Shell-Inline-Muster laden

4.7 Liste sortieren, Duplikate werden entfernt (optional)

Befehl	Funktion
<code>bsafer.sh SORT-PROG <datei></code>	Programm-Liste sortiert anzeigen
<code>bsafer.sh SORT-FOLDER <datei></code>	Ordner-Liste sortiert anzeigen

5. Workflows

5.1 Erstinstallation (Learning Mode)

Learning Mode aktivieren (fest bei Systemstart. Im Kernel aktiviert)

bsafer.sh SLON/SLOFF

System normal nutzen

→ Alle Programme werden protokolliert

→ Skripte (mit/ohne Shebang) werden erfasst

→ Interpreter (ai:) und Skripte (a:) werden getrennt erfasst

→ UID/GID wird automatisch erfasst

→ KEINE Duplikate (intelligente Protokollierung)

Learning Mode deaktivieren

bsafer.sh SLOFF

Liste übernehmen

cat /proc/safer.learning > t

cat /proc/safer.lib.learning > l

cat /proc/safer.module.learning > m

Liste safer.conf erstellen.

Listen laden

bsafer.sh PLIST /etc/safer/safer.conf

bsafer.sh LLIST /etc/safer/safer.conf

bsafer.sh KLIST /etc/safer/safer.conf

Safer aktivieren

bsafer.sh SON

5.2 System-Update

Learning Mode aktivieren

bsafer.sh SLON

Update durchführen

apt update && apt upgrade -y

Learning Mode deaktivieren

bsafer.sh SLOFF

Neue Einträge prüfen (mit User/Group)

Listen neu laden

bsafer.sh PLIST /etc/safer/safer.conf

bsafer.sh LLIST /etc/safer/safer.conf

bsafer.sh KLIST /etc/safer/safer.conf

5.3 Neues Skript ohne Shebang freigeben

#Liste neu laden

bsafer.sh PLIST /etc/safer/safer.conf

5.4 Neuen Interpreter freigeben (ai:)

Liste neu laden

bsafer.sh PLIST /etc/safer/safer.conf

5.5 Konfigurationsdatei-Schutz einrichten (NUR Host!)

Dateien festlegen

edit /etc/safer/safer.conf

→ KONFIG_SFILE:/etc/passwd

→ KONFIG_SFILE:/etc/shadow

SKONFIG-LIST laden

bsafer.sh SKONFIG-LIST /etc/safer/safer.conf

Learning Mode aktivieren

bsafer.sh SLON

System normal nutzen

Learning Mode deaktivieren

bsafer.sh SLOFF

Hashes übernehmen

cat /proc/safer/safer.learning

KONFIG-LIST laden

bsafer.sh KONFIG-LIST /etc/safer/safer.conf

Safer aktivieren

bsafer.sh SON

5.6 Shell-Inline-Muster festlegen

Konfiguration bearbeiten

edit /etc/safer/safer.conf

→ SHELL-INLINE:read

→ SHELL-INLINE:read<

Muster laden

bsafer.sh KONFIG-PATTERN-LIST /etc/safer/safer.conf

Safer aktivieren

bsafer.sh SON

5.7 System sperren (maximale Sicherheit)

bsafer.sh SLOCK

→ Safer kann nicht mehr deaktiviert werden

→ Kein Schalter funktioniert mehr (auch für root) bis Neustart

→ Maximale Sicherheit

6. /proc-Auswertung (Nur User root. Nicht für Group root)

6.1 Blockierte Elemente anzeigen (Abschnitt deny)

```
cat /proc/safer.info          # Blockierte Programme und Skripte)
cat /proc/safer.lib.info      # Blockierte Bibliotheken
cat /proc/safer.module.info   # Blockierte Kernel-Module
```

6.2 Erlaubte Elemente anzeigen

```
cat /proc/safer.info          # Erlaubte Programme und Skripte
cat /proc/safer.lib.info      # Erlaubte Bibliotheken
cat /proc/safer.module.info   # Erlaubte Kernel-Module
```

6.3 Statistik und Konfiguration

```
cat /proc/safer.info          # Statistik, Akt. Konfig, deny
cat /proc/safer.lib.info      # Aktuelle Konfiguration, deny
cat /proc/safer.module.info   # Aktuelle Konfiguration, deny
```

6.4 Logs prüfen

```
dmesg
```

7. Container und Host – Die Unterschiede

Aspekt	Container	Host
Programme (a:, ai:, as;; ga:)	Prüfung gegen Liste	Prüfung gegen Liste
Skripte ohne Shebang	Prüfung gegen Liste	Prüfung gegen Liste
User/Group-Verwaltung	Ja	Ja
Konfigurationsdateien	FEST IM CODE (nur inhaltlich)	ADMIN-FESTGELEGT (Liste mit Hash)
Reaktion bei Manipulation	Blockierung neuer Prozesse	Kontrollierter Shutdown
Admin-Kontrolle	KEINE	VOLLE KONTROLLE

CONTAINER UND HOST – DIE UNTERSCHIEDE

CONTAINER:

- passwd, shadow, group, gshadow → IMMER inhaltlich
- FEST IM CODE
- Admin hat KEINEN Einfluss
- Reaktion: Blockierung neuer Prozesse

HOST:

- Admin wählt die Dateien aus (KONFIG_SFILE:)
- Learning Mode → Hashes (KONFIG:)
- Prüfmethode (Gegen Liste/Hash)
- Reaktion: Kontrollierter Shutdown

9. Glossar

Begriff	Bedeutung
Hash	SHA256-Prüfsumme einer Datei
Shebang	#! am Anfang einer Skript-Datei
Allow Liste	Liste erlaubter Programme
Learning Mode	Automatische Generierung der Allow Liste
TOCTOU	Time-Of-Check, Time-Of-Use (Angriff)
Page Cache	Vom Kernel gecachte Dateiinhalte
initramfs	Initiales RAM-Dateisystem beim Boot
Inhaltliche Prüfung	Prüfung des des Inhalts
KONFIG_SFILE	Festlegen von Konfigurationsdateien (nur Host)
KONFIG	Hash-Prüfung von Konfigurationsdateien (nur Host)
SHELL-INLINE	Muster, die in Shell-Inline Befehlen blockiert werden
EPERM	Fehlercode: Operation nicht gestattet
UID	User ID (Benutzerkennung)
GID	Group ID (Gruppenkennung)
a:	Access (User-basiert)
ai:	Access Interpreter (nur mit Skript)
as:	Access Shell (für Inline-Code)
ga: / ...	Global Access (Gruppen-basiert)
d: / gd:	Deny (User/Ordner)

10. Sicherheitshinweise

11.1 Skripte ohne Shebang

Achtung: Skripte ohne Shebang werden **trotzdem geprüft!** Sie müssen als **a :** in der Allow List stehen. Gilt auch für Java Programme.

11.2 Interpreter-Schutz

Achtung: Ein Interpreter (ai:) darf **NUR in Verbindung mit einem erlaubten Skript (a:)** laufen. Interaktive Aufrufe werden blockiert.

11.3 Learning Mode

Achtung: Der Learning Mode sollte **NUR in kontrollierten Umgebungen** aktiviert werden. Sollte er in einer kompromittierten Umgebung läuft, werden schädliche Programme in die Allow List aufgenommen.

11.4 Lock-Modus

Achtung: Der Lock-Modus (SLOCK) kann **NICHT** rückgängig gemacht werden. Bis zum Neustart. Verwenden Sie ihn NUR, wenn Sie sicher sind, dass die Konfiguration vollständig und korrekt ist.

11.5 Container-Konfiguration

Achtung: Im Container werden die Konfigurations Dateien **fest im Code inhaltlich** geprüft. Der Admin hat **KEINEN Einfluss** darauf. Müste vielleicht im Code angepasst werden.

12. Zusammenfassung

SAFER:

- **KOMBINATION AUS**

HASH-PRÜFUNG, AKTIVE Erkennung Manipulation PAGE CACHE, TOCTOU-Erkennung, SHELL-INLINE-SCHUTZ, BOOT-SCHUTZ (Kernel prüft sich selbst, User kann den Kernel HASH prüfen), LEARNING MODE, BENUTZER-/GRUPPENVERWALTUNG, INTERPRETER Prüfung, SKRIPT-PRÜFUNG (MIT/OHNE SHEBANG) UND KONFIGURATIONS Dateien SCHUTZ.

- **Vorteil**

Prinzip erschließt sich leicht.

User/Group Verwaltung. Übersichtlicher als als MAC/DAC.

Was nicht in der Liste ist, wird nicht ausgeführt.

Was nicht erlaubt ist, wird nicht ausgeführt.

Auch Angriffe zb. über das Netzwerk, werden erkannt.

Spätestens, dann wenn der Angreifer versucht root Rechte zu erhalten oder versucht Daten, auszuleiten.

Eigene Vertrauensliste.

Man weiss, was man erlaubt hat und was nicht.

Permanente Überprüfung der Programme/Libraries/ Kernel Module beim starten gegen eine Liste.

Erstaunlich schnell. Der grosse Block HASH Berechnung relativiert sich schnell.

In der Regel hält der Linux Kernel die Dateien im Page Cache. Dadurch sehr schneller Zugriff. I/O nur beim ersten Laden.

Auch die Konfigurations Dateien wie zb. /etc/passwd werden im „Page Cache“ gehalten.

HASH Berechnung nur mit „Linux Kernel Funktionen“. Dadurch ist gewährleistet, das der Linux Kernel immer die schnellsten Möglichkeiten, die die Hardware bietet, nutzt.

Learning Mode erschließt sich schnell. Auch die Wartung bei Updates ist überschaubar.

Viel Spaß